

手把手之从注册用户接口开始

用户表结构

以下表结构为用户原始表结构，而大家执行过建表语句后，看到数据库中大多为 t_user_0-15 的表名。这是因为，考虑到用户量过大，对用户表进行了分库分表处理。

```
CREATE TABLE `t_user` (  
  `id` bigint(20) unsigned NOT NULL AUTO_INCREMENT COMMENT 'ID',  
  `username` varchar(256) COLLATE utf8mb4_unicode_ci DEFAULT NULL COMMENT '用户名',  
  `password` varchar(512) COLLATE utf8mb4_unicode_ci DEFAULT NULL COMMENT '密码',  
  `real_name` varchar(256) COLLATE utf8mb4_unicode_ci DEFAULT NULL COMMENT '真实姓名',  
  `region` varchar(64) COLLATE utf8mb4_unicode_ci DEFAULT '0' COMMENT '国家/地区',  
  `id_type` int(3) DEFAULT NULL COMMENT '证件类型',  
  `id_card` varchar(256) COLLATE utf8mb4_unicode_ci DEFAULT NULL COMMENT '证件号',  
  `phone` varchar(128) COLLATE utf8mb4_unicode_ci DEFAULT NULL COMMENT '手机号',  
  `telephone` varchar(128) COLLATE utf8mb4_unicode_ci DEFAULT NULL COMMENT '固定电话',  
  `mail` varchar(256) COLLATE utf8mb4_unicode_ci DEFAULT NULL COMMENT '邮箱',  
  `user_type` int(3) DEFAULT NULL COMMENT '旅客类型',  
  `verify_status` int(3) DEFAULT NULL COMMENT '审核状态',  
  `post_code` varchar(64) COLLATE utf8mb4_unicode_ci DEFAULT NULL COMMENT '邮编',  
  `address` varchar(1024) COLLATE utf8mb4_unicode_ci DEFAULT NULL COMMENT '地址',  
  `deletion_time` bigint(20) DEFAULT '0' COMMENT '注销时间戳',  
  `create_time` datetime DEFAULT NULL COMMENT '创建时间',  
  `update_time` datetime DEFAULT NULL COMMENT '修改时间',  
  `del_flag` tinyint(1) DEFAULT NULL COMMENT '删除标识',  
  PRIMARY KEY (`id`),  
  UNIQUE KEY `idx_username` (`username`,`deletion_time`) USING BTREE  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='用户表';
```

分库分表策略

用户数据分库分表相关文档参考下述文档：

[用户数据如何实现分片](#)

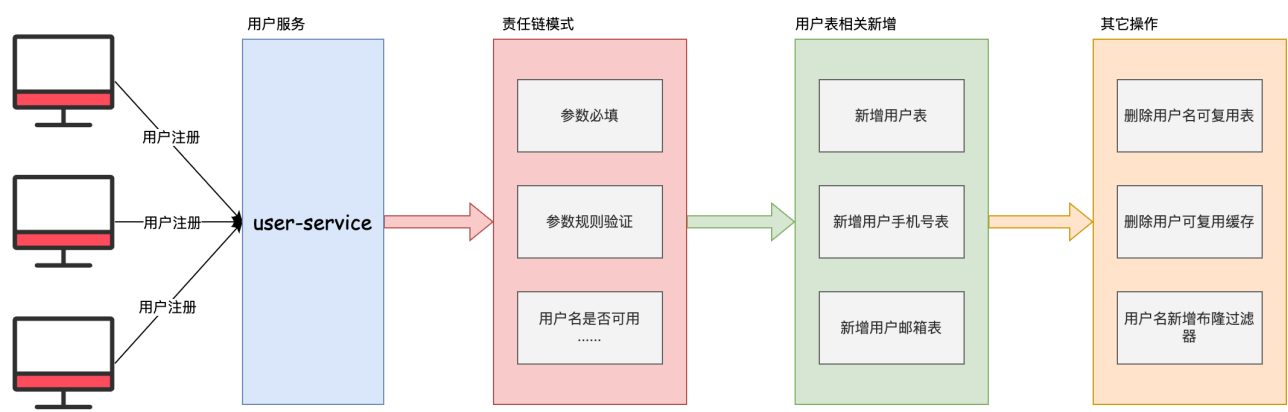
用户注册缓存穿透

用户注册业务缓存击穿场景详细解决方案参考下述文档：

[注册用户如何防止缓存穿透？](#)

用户注册接口

用户注册接口整体流程如下所示。



全部代码如下：

```
@Transactional(rollbackFor = Exception.class)
@Override
public UserRegisterRespDTO register(UserRegisterReqDTO requestParam) {
    // 责任链模式验证注册用户请求参数是否合规
    abstractChainContext.handler(UserChainMarkEnum.USER_REGISTER_FILTER.name(),
requestParam);
    try {
        // 注册用户信息
        int inserted = userMapper.insert(BeanUtil.convert(requestParam, UserDO.class));
        if (inserted < 1) {
            throw new ServiceException(USER_REGISTER_FAIL);
        }
    } catch (DuplicateKeyException dke) {
        log.error("用户名 [{}] 重复注册", requestParam.getUsername());
        throw new ServiceException(PHONE_REGISTERED);
    }
}
```

```
UserPhoneDO userPhoneDO = UserPhoneDO.builder()
    .phone(requestParam.getPhone())
    .username(requestParam.getUsername())
    .build();

try {
    // 注册用户手机号信息
    userPhoneMapper.insert(userPhoneDO);
} catch (DuplicateKeyException dke) {
    log.error("用户 [{}] 注册手机号 [{}] 重复", requestParam.getUsername(),
requestParam.getPhone());
    throw new ServiceException(PHONE_REGISTERED);
}

if (StrUtil.isNotBlank(requestParam.getMail())) {
    UserMailDO userMailDO = UserMailDO.builder()
        .mail(requestParam.getMail())
        .username(requestParam.getUsername())
        .build();

    try {
        // 注册用户邮箱信息
        userMailMapper.insert(userMailDO);
    } catch (DuplicateKeyException dke) {
        log.error("用户 [{}] 注册邮箱 [{}] 重复", requestParam.getUsername(),
requestParam.getMail());
        throw new ServiceException(MAIL_REGISTERED);
    }
}

String username = requestParam.getUsername();
// 删除用户可复用数据
userReuseMapper.delete(Wrappers.update(new UserReuseDO(username)));
StringRedisTemplate instance = (StringRedisTemplate) distributedCache.getInstance();
// 删除用户可复用缓存数据
instance.opsForSet().remove(USER_REGISTER_REUSE_SHARDING + hashShardingIdx(username),
username);
// 增加已存在用户名布隆过滤器
userRegisterCachePenetrationBloomFilter.add(username);
return BeanUtil.convert(requestParam, UserRegisterRespDTO.class);
}
```

责任链模式

如果大家对责任链模式不熟悉，可以查看以下两篇文章。一篇讲什么是责任链，另一篇讲如何抽象。

[责任链模式重构复杂业务场景](#)

[死磕设计模式之如何抽象责任链模式](#)

12306 系统中，我们定义责任链模式分为三步，确定方法执行入参、定义当前业务责任链接口以及具体实施验证责任链执行器。

1) 确定方法执行入参。

因为我们是用户注册接口，验证的也是用户提交的数据，直接复用该实体即可。

```
package org.opengooify.index12306.biz.userservice.dto.req;

import lombok.Data;

/**
 * 用户注册请求参数
 *
 * @公众号：马丁玩编程，回复：加群，添加马哥微信（备注：12306）获取项目资料
 */
@Data
public class UserRegisterReqDTO {

    /**
     * 用户名
     */
    private String username;

    /**
     * 密码
     */
    private String password;

    /**
     * 真实姓名
     */
}
```

```
private String realName;

/**
 * 证件类型
 */
private Integer idType;

/**
 * 证件号
 */
private String idCard;

/**
 * 手机号
 */
private String phone;

/**
 * 邮箱
 */
private String mail;

/**
 * 旅客类型
 */
private Integer userType;

/**
 * 审核状态
 */
private Integer verifyState;

/**
 * 邮编
 */
private String postCode;

/**
 * 地址
 */
private String address;
```

```

/**
 * 国家/地区
 */
private String region;

/**
 * 固定电话
 */
private String telephone;
}

```

2) 定义业务责任链接口。

```

package org.opengoofy.index12306.biz.userservice.service.handler.filter.user;

import org.opengoofy.index12306.biz.userservice.common.enums.UserChainMarkEnum;
import org.opengoofy.index12306.biz.userservice.dto.req.UserRegisterReqDTO;
import
org.opengoofy.index12306.framework.starter.designpattern.chain.AbstractChainHandler;

/**
 * 用户注册责任链过滤器
 *
 * @公众号：马丁玩编程，回复：加群，添加马哥微信（备注：12306）获取项目资料
 */
public interface UserRegisterCreateChainFilter<T extends UserRegisterReqDTO> extends
AbstractChainHandler<UserRegisterReqDTO> {

    @Override
    default String mark() {
        return UserChainMarkEnum.USER_REGISTER_FILTER.name();
    }
}

```

当构建责任链时，为什么要定义具体业务责任链接口而不是直接实现抽象责任链类 `AbstractChainHandler` 呢？

虽然直接实现 `AbstractChainHandler` 是可行的，但这种方式会导致两个问题：

1. 冗余标识：所有责任链处理器都需要实现 `mark` 接口来标识当前类用于处理哪种业务。这样做会增加冗余代码，因为每个处理器类都需要实现相同的接口来提供标识信息。
2. 可读性降低：当我们想查看项目中有多少责任链业务处理时，可能会看到大量的具体实现类。这种情况下，我们更关注责任链的类别而不是具体的实现类。直接实现 `AbstractChainHandler` 会导致责任链类别信息被掩盖，降低了代码的可读性。

因此，为了提高代码的可读性和可维护性，以及避免冗余的标识信息，我们采用定义责任链接口的方式。通过定义责任链接口，每个处理器只需实现自己负责的逻辑，而责任链的类别信息可以通过接口的命名和文档来清晰地体现。

这样一来，我们可以更方便地了解责任链的结构和组成，使代码更加简洁、清晰、易于维护和扩展。

以 `List` 接口举例，当大家点开 `List` 实现类的时候，是不是发现根本就没有可用信息。

```
public interface List<E> extends Collection<E> {  
  
    © AbstractImmutableList in ImmutableCollections (java.util)  
    © AbstractImmutableList (org.apache.calcite.runtime)  
    © AbstractLinkedList (org.apache.commons.collections.list)  
    © AbstractLinkedList (org.apache.commons.collections4.list)  
    © AbstractList (java.util)  
    © AbstractList (kotlin.collections)  
    © AbstractListDecorator (org.apache.commons.collections.list)  
    © AbstractListDecorator (org.apache.commons.collections4.list)  
    © AbstractListWrapper in Lists (com.alibaba.nacos.shaded.com.g  
    © AbstractListWrapper in Lists (com.google.common.collect)  
    © AbstractListWrapper in Lists (org.apache.curator.shaded.com.  
    © AbstractMutableList (kotlin.collections)  
    © AbstractObjectList in XSSimpleTypeDecl (com.sun.org.apache.x  
    © AbstractPartition (io.micrometer.core.instrument.util)  
    © AbstractProtobufList (com.alibaba.nacos.shaded.com.google.pr  
    © AbstractProtobufList (com.google.protobuf)  
    © AbstractProtobufList (com.google.protobuf)
```

而我们用了这种具体业务接口继承自 `AbstractChainHandler` 就能很好解决这个问题。可以非常便于知道，用户注册责任链处理器都有哪些实现。

因此，定义责任链接口是一种更优雅、灵活且可扩展的设计方式，有助于提高代码的可维护性和可扩展性。


```

/**
 * 用户注册责任链过滤器
 *
 * @公众号：马丁玩编程，回复：加群，添加马哥微信（备注：12306）获取项目资料
 */
public interface UserRegisterCreateChainFilter<T extends UserRegisterReqDTO>

Choose Implementation of UserRegisterCreateChainFilter

  (C) UserRegisterCheckDeletionChainHandler (org.opengoofy.index12306.biz.userservice)
  (C) UserRegisterHasUsernameChainHandler (org.opengoofy.index12306.biz.userservice)
  (C) UserRegisterParamNotNullChainHandler (org.opengoofy.index12306.biz.userservice)

```

3) 定义责任链业务具体处理器。

在定义责任链处理器时，需要注意使用 `getOrder` 排序接口来决定组件的执行顺序。通常情况下，处理效率高且在内存中执行的验证策略应该优先执行，而需要涉及交互操作（例如 Redis 等）的处理策略则放在后面执行。

举例来说，假设用户没有传递身份证号，在这种情况下，首先执行验证用户名的操作是非常浪费的，因为后续还需要验证参数必填性，这样就多了一次查询缓存的无用耗时。尽管性能损耗可能不会很高，但这种无用的损耗应该尽量避免。

因此，通过合理地使用 `getOrder` 排序接口，我们可以优化责任链的执行顺序，使得处理效率高的操作优先执行，避免不必要的性能损耗，从而提升整体处理性能和效率。

```

// 验证参数必填
@Component
public final class UserRegisterParamNotNullChainHandler implements
    UserRegisterCreateChainFilter<UserRegisterReqDTO> {

    @Override
    public void handler(UserRegisterReqDTO requestParam) {
        if (Objects.isNull(requestParam.getUsername())) {
            throw new ClientException(UserRegisterErrorCodeEnum.USER_NAME_NOTNULL);
        } else if (Objects.isNull(requestParam.getPassword())) {
            throw new ClientException(UserRegisterErrorCodeEnum.PASSWORD_NOTNULL);
        } else if (Objects.isNull(requestParam.getPhone())) {
            throw new ClientException(UserRegisterErrorCodeEnum.PHONE_NOTNULL);
        }
    }
}

```

```

        } else if (Objects.isNull(requestParam.getIdType())) {
            throw new ClientException(UserRegisterErrorCodeEnum.ID_TYPE_NOTNULL);
        } else if (Objects.isNull(requestParam.getIdCard())) {
            throw new ClientException(UserRegisterErrorCodeEnum.ID_CARD_NOTNULL);
        }
        // xxx 这里应该把所有用户注册入参校验必填，因为重复工作量所以暂时验证上述这些
    }

    @Override
    public int getOrder() {
        return 0;
    }
}

// 验证用户名是否可用
@Component
@RequiredArgsConstructor
public final class UserRegisterHasUsernameChainHandler implements
    UserRegisterCreateChainFilter<UserRegisterReqDTO> {

    private final UserLoginService userLoginService;

    @Override
    public void handler(UserRegisterReqDTO requestParam) {
        if (!userLoginService.hasUsername(requestParam.getUsername())) {
            throw new ClientException(UserRegisterErrorCodeEnum.HAS_USERNAME_NOTNULL);
        }
    }

    @Override
    public int getOrder() {
        return 1;
    }
}

// 验证证件号是否多次注销，如果是的话加入黑名单
@Component
@RequiredArgsConstructor
public final class UserRegisterCheckDeletionChainHandler implements
    UserRegisterCreateChainFilter<UserRegisterReqDTO> {

```

```
private final UserService userService;

@Override
public void handler(UserRegisterReqDTO requestParam) {
    Integer userDeletionNum =
userService.queryUserDeletionNum(requestParam.getIdType(), requestParam.getIdCard());
    if (userDeletionNum >= 5) {
        throw new ClientException("证件号多次注销账号已被加入黑名单");
    }
}

@Override
public int getOrder() {
    return 2;
}
}
```

用户表相关

有同学对于新增用户时为什么需要同时操作用户表、用户手机号表和用户邮箱表表示疑问。

在回答之前，我们先来查看 12306 登录功能的原型截图及其对应的功能。



欢迎登录

账号登录



面容 ID 登录

用户

用户名/邮箱/手机号码

密码

登录密码

[忘记密码?](#)

登录

注册

在登录功能中，用户一栏明确标出可以使用用户名、邮箱或手机号中的任意一个搭配密码进行登录。需要强调的是，在分库分表中，我们是通过用户名进行分片的。因此，如果在查询用户信息时不带用户名，将会触发读扩散问题。

引用之前写过的一段话，来详细说明分库分表后，不带用户分片键查询的危害。

由于登录时没有带用户名，导致无法确定用户的分片键，使得系统无法直接锁定用户的数据位于哪个数据库或者哪张表中。为了找到用户的数据，只能对全部的数据库和表进行扫描查询，这就造成了所谓的“读请求扩散”问题。

也就是说，原本读请求可以直接定位到某个数据库某张表，现在却要多处查询，无疑大大增加了系统的查询负载。

一旦出现了读请求扩散问题，势必会导致用户的登录请求响应时间变长，严重的话还可能造成登录超时。

为了解决这个问题，我们引入了两张路由表：用户手机号表和用户邮箱表。这些表的核心字段是手机号和邮箱，以及它们对应的用户名。通过这样的设计，我们能够在用户登录时，灵活地使用手机号、邮箱或用户名来进行认证。

```
CREATE TABLE `t_user_mail` (  
  `id` bigint(20) NOT NULL AUTO_INCREMENT COMMENT 'ID',  
  `username` varchar(256) COLLATE utf8mb4_unicode_ci DEFAULT NULL COMMENT '用户名',  
  `mail` varchar(256) COLLATE utf8mb4_unicode_ci DEFAULT NULL COMMENT '邮箱',  
  `deletion_time` bigint(20) DEFAULT '0' COMMENT '注销时间戳',  
  `create_time` datetime DEFAULT NULL COMMENT '创建时间',  
  `update_time` datetime DEFAULT NULL COMMENT '修改时间',  
  `del_flag` tinyint(1) DEFAULT NULL COMMENT '删除标识',  
  PRIMARY KEY (`id`),  
  UNIQUE KEY `idx_mail` (`mail`,`deletion_time`) USING BTREE  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='用户邮箱表';
```

```
CREATE TABLE `t_user_phone` (  
  `id` bigint(20) NOT NULL AUTO_INCREMENT COMMENT 'ID',  
  `username` varchar(256) COLLATE utf8mb4_unicode_ci DEFAULT NULL COMMENT '用户名',  
  `phone` varchar(128) COLLATE utf8mb4_unicode_ci DEFAULT NULL COMMENT '手机号',  
  `deletion_time` bigint(20) DEFAULT '0' COMMENT '注销时间戳',  
  `create_time` datetime DEFAULT NULL COMMENT '创建时间',  
  `update_time` datetime DEFAULT NULL COMMENT '修改时间',  
  `del_flag` tinyint(1) DEFAULT NULL COMMENT '删除标识',  
  PRIMARY KEY (`id`),  
  UNIQUE KEY `idx_phone` (`phone`,`deletion_time`) USING BTREE  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='用户号码表';
```

什么是路由表？

在分库分表中，路由表（Routing Table）是一个用于映射用户数据在分片库和分片表中位置的特殊表。它记录了用户按照特定分片键（例如用户名、邮箱、手机号等）拆分后在哪个分片库的哪张分片表中。

路由表的存在使得系统可以根据不同的登录方式（用户名、邮箱、手机号）来准确地定位到用户数据所在的分片位置，从而能够在查询时快速找到对应的数据。

路由表优点

通过路由表的映射，系统可以快速定位用户数据所在的分片位置，从而实现按照不同的登录方式进行查询。这样，无论用户是通过用户名、邮箱还是手机号登录，系统都能准确地找到对应的数据所在位置，从而提高了查询的效率和准确性。

拿手机号登录方式来说，我们通过手机号定位到路由表中的某一条用户数据，拿到对应的用户名，再根据用户名去查询真实的用户数据。这样就很好避免了读扩散问题。

路由表缺点

当然，路由表也不是万能的，同样存在着一些不好的点，比如：

- 查询性能：在进行用户登录或其他需要根据分片键查找数据的操作时，需要先访问路由表，根据分片键的取值范围定位到对应的分片位置，然后再去实际的分片库和分片表中查询数据。这样的查询过程会引入额外的查询开销，可能影响系统的查询性能。

- **维护成本：**随着系统数据量的增加和业务发展，路由表的大小和复杂度可能会不断增加，维护路由表将会变得越来越复杂。对于大规模分库分表系统，路由表的维护成本可能会很高。

整体来说，为了实现多种方式用户登陆需求，路由表的设计算是比较折中的，也是大规模应用生产系统。

其它操作

在用户注册后，该用户名将不再可用，因此我们需要将其添加到布隆过滤器中，以防止其他人在后续尝试注册时再次使用。

另外，关于用户注册文章中提到的用户名可复用问题，我们特别考虑了这一点。已经注销的用户名需要能够被其他用户再次使用，因此我们对可复用用户名进行了扩展设计。

这意味着，我们需要将对应的缓存和数据库表一并删除。一旦删除完成，后续用户将无法再通过这个扩展点使用该用户名。这样的设计保证了用户名的合理复用，同时确保已注销用户名的信息不会对后续用户产生影响。

文末总结

12306 系统对用户数据进行了分库分表操作，将用户表拆分为多张表，通过用户名进行分片，以应对大量用户数据的存储和查询需求。同时，为了提供更多登录方式的支持，系统扩充了手机号和邮箱登录方式，使用户可以根据任意一种方式搭配密码进行登录，增加了系统的灵活性和用户友好性。

用户注册流程详细描述：包括了注册请求参数合规性验证、用户信息的插入操作，以及手机号和邮箱信息的处理。在用户注册后，系统将注册的用户名添加到布隆过滤器中，防止后续再被其他人注册。为了解决已注销用户名的可复用问题，相关缓存和数据库表也会被删除，确保用户名的合理复用和数据的一致性。

在分库分表的实现中，引入了两张路由表：用户手机号表和用户邮箱表。这些表记录了用户数据按照特定分片键进行拆分后的位置信息，从而在查询时能够准确地定位到用户数据所在的分片位置，避免了读扩散问题，提高了查询性能和系统的响应速度。

尽管分库分表带来了查询性能的提升和数据存储的分散，但也面临着一些挑战，例如维护路由表的成本随着数据量增加而增加，查询时需要额外访问路由表，可能影响查询性能。因此，在系统设计中需要综合考虑分库分表带来的利弊，并做出合理的权衡和优化。

总体而言，通过分库分表和扩充登录方式，12306 系统有效地应对了海量用户数据和多样化登录需求，提升了系统的可扩展性和性能，为用户提供了更好的使用体验。不过在大规模分库分表系统中，仍需注意维护成本和查询性能等方面的优化，以确保系统持续稳定高效运行。

更新: 2023-09-24 11:01:47

原文: <<https://www.yuque.com/magestack/12306/bhzetv04b8ydt1kg>>